

A Computer Vision Platform for Sustainable Agriculture in Ghana

PRESENTER · EPIGOS AI

HOSTED BY · GHANA AI NETWORK (GAIN)

BACKED BY

NVIDIA INCEPTION · AWS STARTUPS

YOUR HOST



Philip Adzanoukpe

FOUNDER • EPIGOS AI

Software engineer and AI expert dedicated to leveraging technology to solve real-world challenges — leading Epigos AI's mission to transform model development across industries.

01 Honorary Visiting Fellow · City St. George's, University of London

02 Distinguished Academic STEM Alumni Award

03 MSc in Data Science · City St. George's, University of London

WHAT WE'LL COVER

Agenda

01 Introducing Epigos AI

03 The Vision Pipeline

05 Training & Evaluation

02 Why CV for Agriculture

04 Dataset Preparation

06 Deployment & Live Demo

WHAT WE DO

A no-code platform to **annotate data, train** computer vision models, **and deploy** them anywhere — built in Ghana, used globally.

DATASET MANAGEMENT

IMAGE ANNOTATION

AUTO-LABEL

HOSTED GPU TRAINING

REST API & SDKS

EDGE DEPLOYMENT

THE OPPORTUNITY

Vision models turn every camera into an agronomist.

40%

of crop yield in sub-Saharan Africa is lost to **diseases & pests** that can be caught early with image-based diagnostics.

01 Early disease detection

Identify blight, mosaic, and pod rot from a single leaf or fruit photo.

02 Yield & quality estimation

Count fruit, grade ripeness, flag defects before harvest.

03 Pest surveillance

Automate trap counts and larvae census at field scale.

MODELS IN PRODUCTION

From cocoa farms to clinics.



COCOA • GHANA

Cocoa disease detection

Classifies black pod, swollen shoot, and healthy fruit from phone-camera images captured by field officers.



ENTOMOLOGY

Insect larvae identification & count

Replaces manual counts in pest surveillance workflows — one image, 300 larvae detected automatically.



HEALTHCARE • PILOT

Ocular disease detection

Pilot with a clinical partner in South Africa — same platform applied to retinal imagery for early screening.

BEYOND AGRICULTURE

One platform. Many sectors.

Object detection is a general tool. Wherever a camera sees something worth counting, sorting, or flagging — the pipeline is the same.



01 • AGRICULTURE

Disease, pest & yield

Diagnose crop disease, count fruit and livestock, grade produce — from a phone in the field.



02 • RETAIL

Shelf & stock intelligence

Audit planograms, flag out-of-stock SKUs, and measure share-of-shelf from store photos.



03 • MANUFACTURING

Defect & quality control

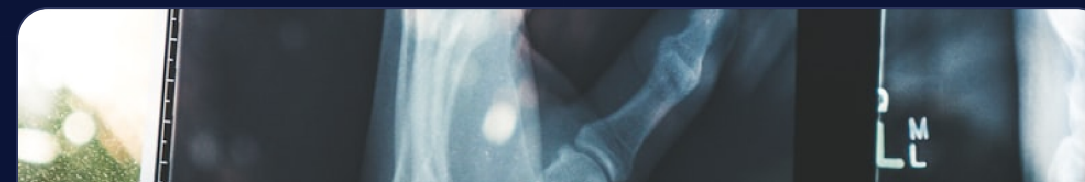
Catch scratches, misalignments and missing components on the line at full production speed.



04 • LOGISTICS

Warehouse & tracking

Count pallets, read labels, and monitor loading bays for throughput and safety compliance.



05 • HEALTHCARE

Assistive screening

Highlight findings on X-ray, retinal and pathology imagery to support clinician triage.



06 • MOBILITY

Vehicles & smart cities

Count vehicles, monitor road safety, and read signs — for transport planners and ADAS.

END-TO-END

The computer vision pipeline.

01 · COLLECT



Collect

Gather diverse, real-world images representative of deployment conditions.

02 · ANNOTATE



Annotate

Draw bounding boxes and assign class labels — or use auto-label.

03 · TRAIN



Train

Fit a model on hosted GPUs with augmentation and smart defaults.

04 · EVALUATE



Evaluate

Measure precision, recall, and mAP against held-out test data.

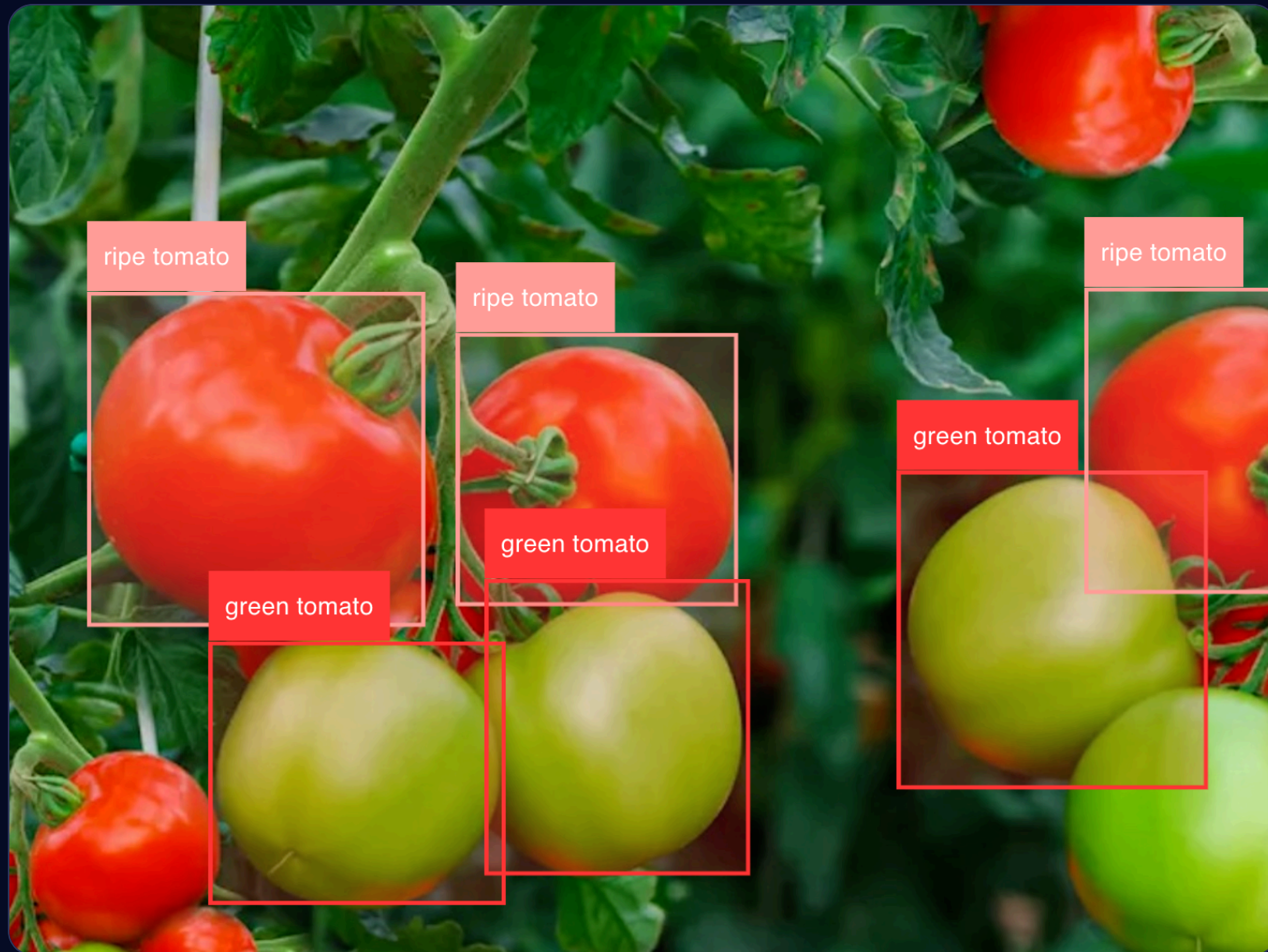
05 · DEPLOY



Deploy

Serve via API, export to PyTorch, or ship to edge as ONNX.

Dataset preparation.



GATHER

Diverse real-world images — different lighting, angles, growth stages and disease severity.

LABEL

Bounding boxes + class names per object. One image can have many boxes.

TOOLS

Bounding box, polygon, and auto-label to pre-fill annotations.

QUALITY

Review workflow catches mislabels before they reach training.

VOLUME

Start with ~200 images per class; grow from there.

BEFORE TRAINING

Split the data. Augment the rest.

Train / Validation / Test

70%

TRAIN

20%

VALIDATION

10%

TEST ·
HELD OUT

Train updates the model. Validation tunes it. Test is used once, at the end, to report honest performance.

Augmentation · one image, many views

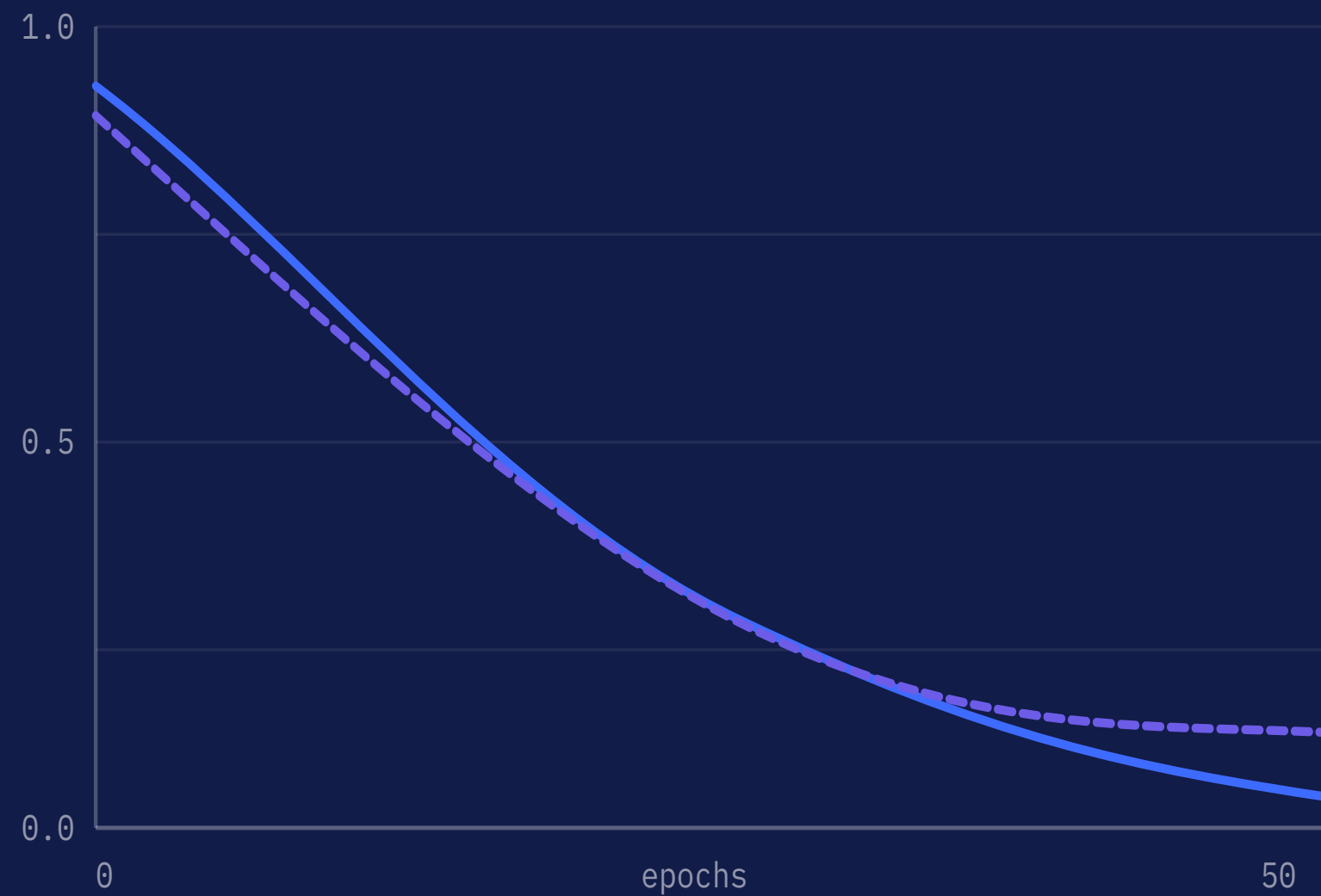


THE TRAINING LOOP

Model learns. Loss drops. Iterate.

Loss over epochs

— Train — Validation



Training recipe

Architecture	YOLO · object detection
Hardware	Hosted NVIDIA GPUs
Batch size	16
Epochs	50 · early-stop on val loss
Optimizer	AdamW · LR 1e-3
Watch for	Train ↓ · Val ↓ · gap small

HOW GOOD IS THE MODEL?

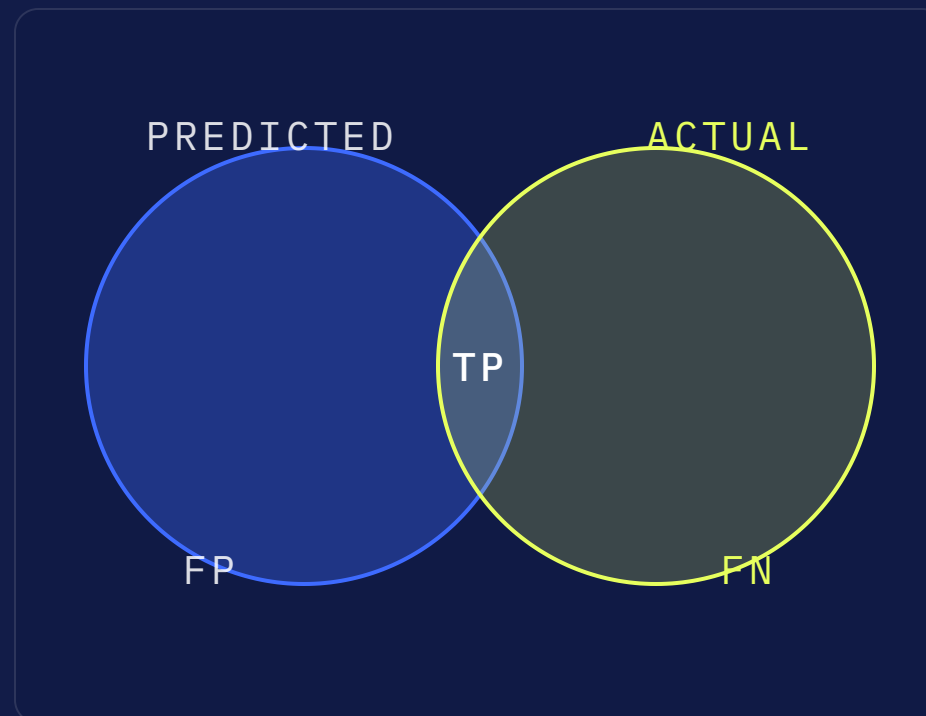
Precision, recall, mAP.

01 · Precision

Precision

$$TP / (TP + FP)$$

Of everything the model flagged as diseased, how much was actually diseased?

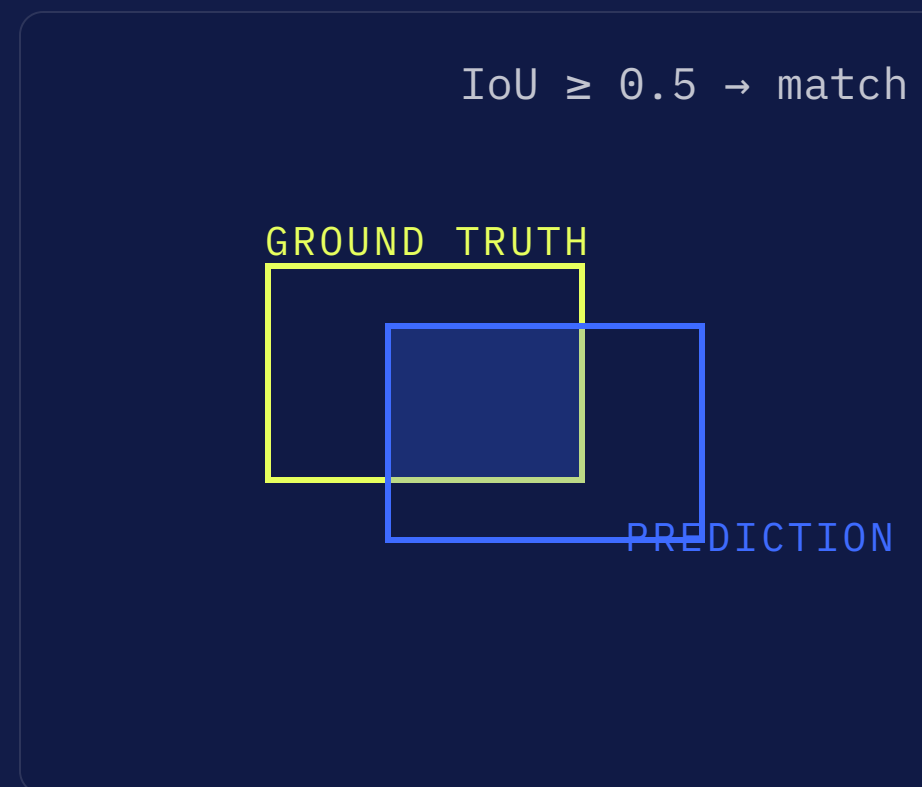


02 · Recall

Recall

$$TP / (TP + FN)$$

Of every diseased object that was truly there, how much did the model catch?



03 · mAP

mean AP

mean of AP across classes

Area under the precision–recall curve, averaged across every class. One number to rule them all.



Three ways to deploy.

01 • HOSTED

REST API

Call your model from anywhere. Send an image, get back detections. Zero infrastructure to manage.

```
# Python SDK
from epigos import Epigos
client = Epigos("api_key")
model =
client.object_detection("model_id")
result = model.detect("leaf.jpg")
```

02 • EXPORT

PyTorch weights

Download the full model as a .pt checkpoint for further research, fine-tuning, or custom training loops.

```
# Load locally
from ultralytics import YOLO
model = YOLO("tomato.pt")
results = model("leaf.jpg")
```

03 • EDGE

ONNX runtime

Export to ONNX and run on edge devices — NVIDIA Jetson, Raspberry Pi, mobile, browser. Same model, any hardware.

```
# Edge inference
import onnxruntime as ort
sess =
ort.InferenceSession("tomato.onnx")
out = sess.run(None, {"input": img})
```

LIVE DEMO

Now let's build it **live**.

Tomato disease object detection — upload, annotate, train,
evaluate, deploy.

FOLLOW ALONG → [APP.EPIGOS.AI](https://app.epigos.ai) [DOCS.EPIGOS.AI](https://docs.epigos.ai)